

SECTION 2 PROGRAMMING MODEL

This section describes the registers of the MC68881/MC68882 (FPCP) programming model. The notation used to refer to the registers conforms to the Freescale assembler syntax. The programming model for the MC68882 is identical to that for the MC68881.

Figure 2-1 is a pictorial representation of the registers in the FPCP programming model. The following paragraphs describe each group of registers.

2.1 FLOATING-POINT DATA REGISTERS

The eight 80-bit floating-point data registers (FP7–FP0) are analogous to the integer data registers (D7–D0) of all M68000 Family processors. Floating-point data registers always contain extended precision numbers. The data format used is identical to the extended precision data format described in Table 3-3, except that the reserved (unused) 16 bits are deleted from the table. All external operands, regardless of the data format, are converted to extended precision values before being used in any calculation or stored in a floating-point data register.

A reset function or a restore operation of the null state sets FP7–FP0 to positive nonsignaling not-a-numbers (NaNs), described in 3.2.5 Not-A-Numbers.

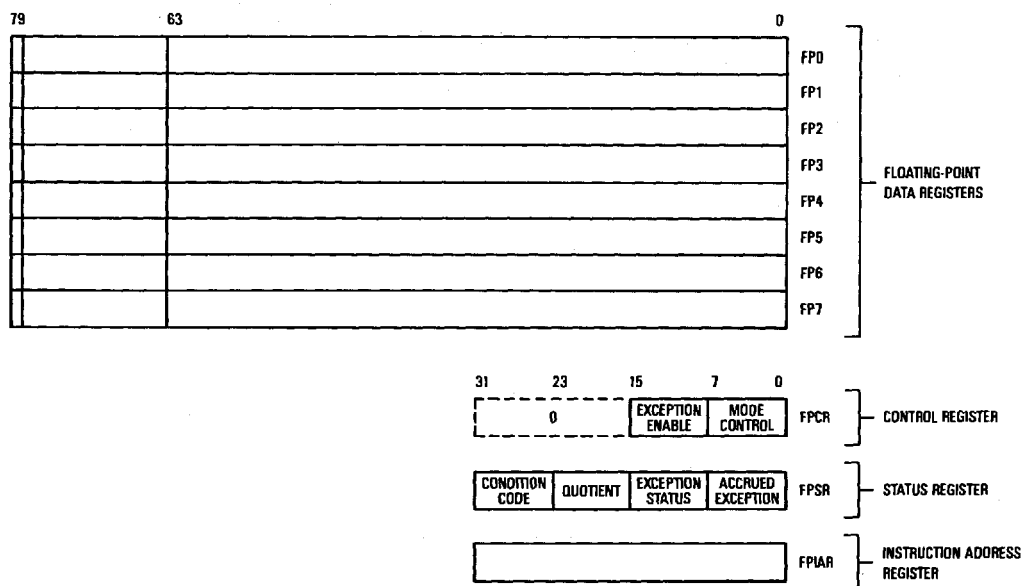


Figure 2-1. MC68881/MC68882 Programming Model

The 32-bit floating-point control register (FPCR) contains an exception enable byte that enables/disables traps for each class of floating-point exceptions and a mode byte that sets the user selectable modes.

The control register can be read or written to by the user. Bits 31–16 are reserved for future definition by Freescale. These bits are always read as zero and are ignored during write operations (but should be zero for future compatibility). This register is cleared by the reset function or a restore operation of the null state. When cleared, this register provides the IEEE standard defaults.

2.2.1 FPCR Exception Enable Byte

One of the bits of the exception enable byte (ENABLE), shown in Figure 2-2, corresponds to each floating-point exception class. The user can separately enable traps for each class of floating-point exceptions.

If a bit in the status register exception byte is set by the FPCP and the corresponding bit in the control register ENABLE byte is also set, an exception is signaled. The address of the exception handler is derived from the vector address corresponding to the exception. When a user writes to the control register ENABLE byte that enables a class of floating-point exceptions, a previously generated floating-point exception does not cause a trap to be taken regardless of the value in the status register exception byte.

The eight floating-point exception classes shown in Figure 2-2 are described in greater detail in **SECTION 6 EXCEPTION PROCESSING**. Note that the bits in the FPSR exception byte and the FPCR enable byte occupy the same positions within each byte.

In a few cases, dual and triple exceptions can be generated by a single instruction execution. When multiple exceptions occur with traps enabled for more than one exception class, the highest priority exception is reported; the lower priority exceptions are never reported or taken. The exception handler routine must check for multiple exceptions. The bits of the

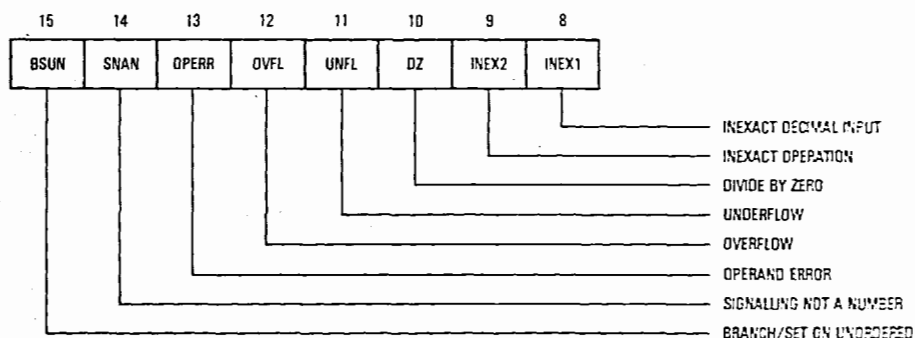


Figure 2-2. MC68881/MC68882 FPCR Exception Enable Byte

- OVFL and INEX1
- OPERR and INEX2
- OPERR and INEX1
- OVFL and INEX2 and/or INEX1
- UNFL and INEX2 and/or INEX1
- INEX2 AND INEX1

2.2.2 FPCR Mode Control Byte

The mode control byte (MODE), shown in Figure 2-3, controls the user-selectable rounding modes and rounding precisions. A zero in this byte selects the IEEE defaults.

The rounding mode specifies how inexact results are rounded. "Round to the nearest" specifies that the nearest number to the infinitely precise result should be selected as the rounded value. In case of a tie, the even result is selected. "Round towards zero" truncates the result. "Round towards plus infinity" always rounds numbers towards plus infinity. "Round toward minus infinity" always rounds numbers towards minus infinity. See 6.1.7 **Inexact Result** for a detailed description of the rounding algorithm used.

The rounding precision selects where rounding of the mantissa occurs. For extended precision, the result is rounded to a 64-bit boundary. A single precision result is rounded to a 24-bit boundary, and a double precision result is rounded to a 53-bit boundary.

The single and double rounding precisions are provided for emulation of machines that only support those precisions. When the FPCR performs any operation, the calculation is carried out using extended precision inputs and the intermediate result is calculated as if to produce infinite precision. After the calculation is complete, this intermediate result is rounded to the selected precision and stored in the destination.

If the destination is a floating-point data register, the stored value is in the extended precision format rounded to the precision specified by the PREC bits. Thus, all mantissa bits beyond the selected precision are zero after the rounding operation. Also, if the single

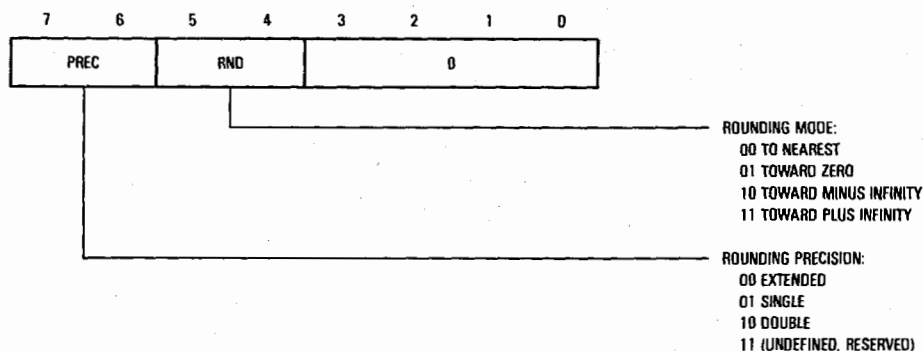


Figure 2-3. MC68881/MC68882 FPCR Mode Control Byte

When single or double precision mode is selected, the exponent value is in the correct range for the single or double precision format (although it is stored in extended precision format). An important exception to this rule is for the FSGLDIV and FSGLMUL instructions. Regardless of the precision specified by the PREC bits, these instructions round the result mantissa to single precision and generate an extended precision exponent which may be out of range for a single precision number.

If the destination is a memory location, the PREC bits are ignored. In this case, a number in the extended precision format is taken from the source floating-point data register, rounded to the destination format precision, and written to memory.

The execution speed of all instructions is degraded significantly when single and double precision rounding modes are used. Because these modes are intended to be used for emulation, this reduction is not detrimental. When operating in these modes, the FPCP produces the same result as any other machine that conforms to the IEEE standard without supporting extended precision calculations. However, the result obtained by performing a series of operations with single or double precision rounding may not be the same as the result of performing the same operations in extended precision and storing the final result in the single or double precision format.

2.3 FLOATING-POINT STATUS REGISTER

The floating-point status register (FPSR) contains a floating-point condition code byte, a floating-point exception status byte, quotient bits, and a floating-point accrued exception byte. All bits in the FPSR can be read or written by the user. Execution of most floating-point instructions modifies this register.

The reset function or a restore operation of the null state clears the FPSR.

2.3.1 FPSR Floating-Point Condition Code Byte

The floating-point condition code byte (FPCC), shown in Figure 2-4, contains four condition code bits that are set at the end of all arithmetic instructions involving the floating-point data registers. The FMOVE FPM,<ea>, move multiple floating-point data register, and move system control register instructions do not affect the FPCC.

The operation result data type determines how the four condition code bits are set. Table 2-1 lists the condition code bit settings for each result data type. Because of the mutually exclusive nature of the data types described by the condition code bits, the FPCP generates

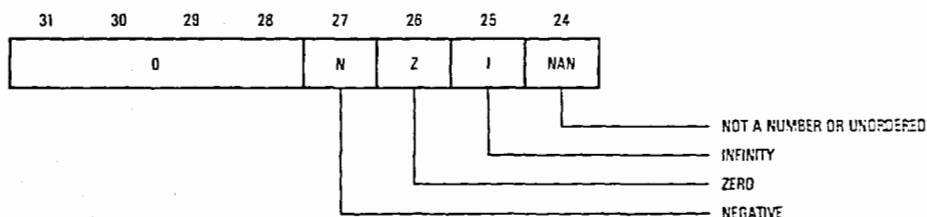


Figure 2-4. MC68881/MC68882 FPSR Condition Code Byte

Table 2-1. Condition Code versus Result Data Type

N	Z	I	NAN	Result Data Type
0	0	0	0	+ Normalized or Denormalized
1	0	0	0	- Normalized or Denormalized
0	1	0	0	+0
1	1	0	0	-0
0	0	1	0	+ Infinity
1	0	1	0	- Infinity
0	0	0	1	+ NAN
1	0	0	1	- NAN

only eight of the 16 possible combinations. Loading the FPCC byte with one of the other condition code bit combinations and executing a conditional instruction may produce an unexpected branch condition.

The IEEE standard defines the following four conditions and only requires the generation of the condition codes as a result of a floating-point compare operation. In addition to this requirement, the FPCP can test these conditions at the end of any operation affecting the condition codes.

- EQ —Equal To
- GT —Greater Than
- LT —Less Than
- UN —Unordered

An unordered condition occurs when one or both of the operands in a floating-point compare operation is a NAN. For purposes of the floating-point conditional branch, set byte on condition, decrement and branch on condition, and trap-on condition instructions, the FPCP logically combines the four condition codes to form the IEEE conditions according to the following equations:

- EQ = Z
- GT = $\overline{N} \vee \overline{NAN} \vee \overline{Z}$
- LT = $NAN \vee \overline{N} \vee \overline{Z}$
- UN = NAN

where:

- " $\wedge$ " = Logical AND
- " $\vee$ " = Logical OR

Note that the setting of the FPCP condition codes is independent of the operation executed; the condition codes only indicate the data type of the result generated. Unlike other M68000 condition codes, the IEEE defined conditions can always be derived from the data type of the result. The setting of the M68000 integer condition codes is dependent upon the operation executed as well as the result.

To aid programmers of floating-point subroutine libraries, the FPCP implements the four previously described floating-point condition code bits in hardware instead of the four IEEE defined conditions. The IEEE conditions are derived by an instruction when needed. For example, the programmer of a complex arithmetic multiply subroutine usually prefers to handle "special" data types such as zeros, infinities, or NANs, separately from "normal" data types. The FPCP condition codes allow users to efficiently detect and handle these "special" values.

The quotient byte (see Figure 2-5) is set at the completion of the modulo (FMOD) or IEEE remainder (FREM) instructions. This byte contains the seven least significant bits of the quotient (unsigned) and the sign of the entire quotient.

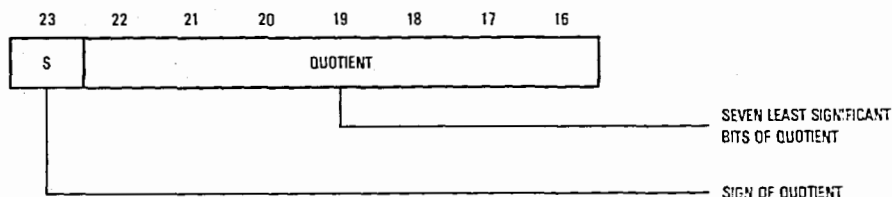


Figure 2-5. MC68881/MC68882 FPSR Quotient Byte

The quotient bits can be used in argument reduction for transcendental functions and other functions. For example, seven bits are more than enough to determine the quadrant of a circle in which an operand resides. The quotient bits remain set until they are cleared by the user, or until another FMOD or FREM instruction is executed.

2.3.3 FPSR Exception Status Byte

The exception status byte (EXC), shown in Figure 2-6, contains a bit for each floating-point exception that may have occurred during the most recent arithmetic instruction or move operation. This byte is cleared by the FPCP at the start of most operations; operations that cannot generate any floating-point exceptions (the FMOVE and FMOVE control register instructions) do not clear this byte. This byte can be used by an exception handler to determine which floating-point exception(s) caused a trap.

If a bit is set by the FPCP in the EXC byte and the corresponding bit in the ENABLE byte is also set, an exception is signaled to the main processor. When a floating-point exception is detected by the FPCP, the corresponding bit in the EXC byte is set, even if the trap for

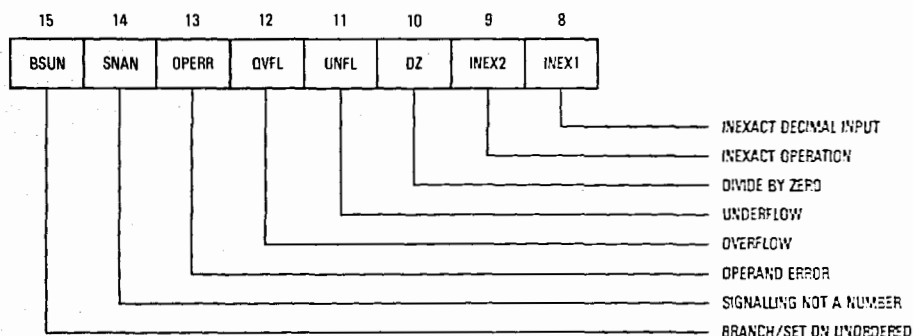


Figure 2-6. MC68881/MC68882 FPSR Exception Status Byte

ception class is disabled. (A user write operation to the status register, which sets the EXC byte, does not cause a trap to be taken regardless of the value in the ENABLE byte.)

Note that the bits in the status EXC byte and control ENABLE byte are in the same bit positions within each byte. The eight floating-point exception classes are described in greater detail in **SECTION 6 EXCEPTION PROCESSING**.

2.3.4 FPSR Accrued Exception Byte

The accrued exception byte (AEXC), shown in Figure 2-7, contains the five exception bits required by the IEEE standard for trap disabled operation. These exceptions are logical combinations of the bits in the EXC byte. The AEXC byte contains the history of all floating-point exceptions that have occurred since the user last cleared the AEXC byte. In normal operations, only the user clears this byte (by writing to the status register). The AEXC byte is cleared by the FPCP only by a reset or a restore operation of the null state.

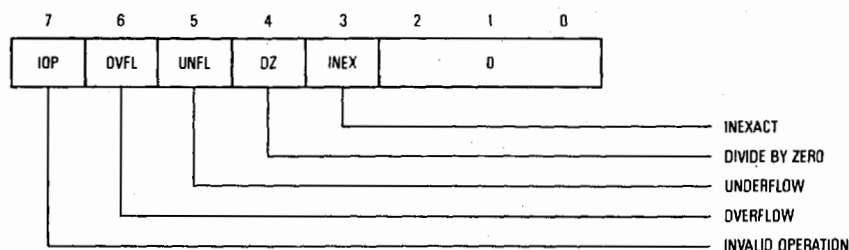


Figure 2-7. MC68881/MC68882 FPSR Accrued Exception Byte

Many users elect to disable traps for all or part of the floating-point exception classes. The AEXC byte is provided to make it unnecessary to poll the EXC byte after each floating-point instruction. At the end of most operations (all but the FMOVE and FMOVE control register instructions), the bits in the EXC byte are logically combined to form an AEXC value that is logically ORed into the existing AEXC byte. This operation creates "sticky" floating-point exception bits in the AEXC byte that the user need poll only once (at the end of a series of floating-point operations, for example).

The setting or clearing of bits in the AEXC byte does not cause the FPCP to take an exception, nor does it prevent taking an exception. The relationship between the bits in the EXC byte and the bits in the AEXC byte is shown by the following equations. These equations apply to setting the AEXC bits at the end of each operation that affects the AEXC byte:

$$\text{AEXC(IOP)} = \text{AEXC(IOP)} \vee \text{EXC(BSUNvSNANvOPERR)}$$

$$\text{AEXC(OVFL)} = \text{AEXC(OVFL)} \vee \text{EXC(OVFL)}$$

$$\text{AEXC(UNFL)} = \text{AEXC(UNFL)} \vee \text{EXC(UNFL} \wedge \text{INEX2)}$$

$$\text{AEXC(DZ)} = \text{AEXC(DZ)} \vee \text{EXC(DZ)}$$

$$\text{AEXC(INEX)} = \text{AEXC(INEX)} \vee \text{EXC(INEX1} \vee \text{INEX2} \vee \text{OVFL)}$$

where:

" $\vee$ " = Logical OR

" $\wedge$ " = Logical AND

A majority of the FPCP instructions operate concurrently with the MC68020/MC68030 (MPU). That is, the MPU can be executing instructions while the FPCP is simultaneously executing a floating-point instruction. Additionally, the MC68882 can execute two floating-point instructions concurrently. As a result of this nonsequential instruction execution, the program counter value stacked by the MPU, in response to an enabled floating-point exception trap may not point to the offending instruction.

For the subset of the FPCP instructions that generate floating-point exception traps, the 32-bit floating-point instruction address (FPIAR) register is loaded with the logical address of an instruction before the instruction is executed (unless all arithmetic exceptions are disabled). This address can then be used by a floating-point exception handler to locate a floating-point instruction that has caused an exception. Since the FPCP FMOVE to/from the FPCR, FPSR, or FPIAR and FMOVEM instructions cannot generate floating-point exceptions, these instructions do not modify the FPIAR. These instructions can be used to read the FPIAR in the trap handler without changing the previous value.

This register is cleared by the reset operation or a restore operation of the null state.